分支与工作树，究竟是什么?

面向 YOLO 剪枝、蒸馏实验的两页总结

分支决定：修改记在哪条版本路线上。
工作树决定：在哪个文件夹里编辑、运行这份代码。

概念	作用	直观理解
Git	管理代码历史，支持比较、合并与恢复。	版本管理系统
提交 Commit	保存纳入提交的文件状态与修改记录。	可追溯的存档点
分支 Branch	指向提交，并随新提交向前推进。	不同实验的修改路线
工作树 Worktree	实际检出、编辑和运行代码的目录。	放代码的实验工作台
GitHub	托管远程 Git 仓库，便于同步与协作。	远程代码仓库

只有分支，与分支配合工作树

只有分支：一个文件夹，在 main 与剪枝分支之间切换；切换时，Git 更新目录中的受控文件。创建分支本身不会增加文件夹。

配合工作树：原目录检出 main，另一个目录检出剪枝分支；两份代码可同时打开，且共用仓库历史。同一个分支通常不能同时检出到两个工作树。

你的实验可以这样摆放

用途	分支示例	工作目录
原始基线	main	原项目目录
单独剪枝	codex/pruning	独立剪枝工作树
单独蒸馏	codex/distillation	独立蒸馏工作树

这些名称和目录是组织示例，不代表本次已创建。若比较剪枝与蒸馏的独立效果，让两条实验路线从同一基线提交出发。

怎样用在你的实验里

01　合并、回退与实验衔接

两种用法都支持合并和恢复历史，因为底层都由 Git 管理。合并针对分支上的改动；恢复版本依据提交记录。工作树提供操作这些代码的目录。

剪枝后再蒸馏：先验证并提交剪枝版本，再以该提交创建 codex/pruning-distill 分支及工作树，用于蒸馏恢复精度。

及时提交才有明确的存档点。未提交的修改、未跟踪或被忽略的数据与权重，不能指望靠 Git 完整恢复。合并代码不会自动融合模型权重。

02　每次实验留下什么

记录代码提交编号、父权重、配置、数据划分、随机种子、依赖版本和评估指标。各实验使用独立结果目录，保持评估口径一致。

工作树隔离代码目录，但不会自动隔离 Python 环境、GPU 或外部输出路径。需要时另建虚拟环境；使用绝对路径保存结果时，尤其要避免互相覆盖。

03　让 Codex 更好地配合你

文件或能力	用途与例子
README.md	项目说明书：目的、安装方法、训练与评估入口。
AGENTS.md	给 AI 的长期项目规则：不覆盖原始权重；先小规模验证；记录实验依据。文件名有 S。
Skill / SKILL.md	可重复使用的任务流程，例如评估模型、对比指标、生成报告。
MCP	连接外部工具和资料的标准方式，例如访问 GitHub 或文档服务。
归档聊天	收起聊天并保留记录，可在设置中恢复；归档本身不会跨账号公开，也不提供账号隔离。

建议学习顺序：提交 → 分支 → 工作树 → 查看改动 → 虚拟环境 → AGENTS.md → Skills。

参考：OpenAI 官方文档（界面入口可能随版本变化）
工作树与分支 · https://learn.chatgpt.com/docs/environments/git-worktrees
AGENTS.md、Skills 与 MCP · https://learn.chatgpt.com/docs/customization/overview
归档聊天与恢复 · https://learn.chatgpt.com/docs/reference/troubleshooting